

Multithreading

In Java Collections, not all collections are thread-safe.

Non-thread-safe collections:

ArrayList, HashMap, and HashSet are not thread-safe. If multiple threads use them at the same time, problems like race conditions can happen.

Thread-safe collections:

1. Legacy classes (old):
Vector and Hashtable are thread-safe because they use synchronized methods internally, but they are slower.
2. Synchronized wrappers:
We can make collections thread-safe using:
`Collections.synchronizedList(new ArrayList<>());`
`Collections.synchronizedMap(new HashMap<>());`
3. Concurrent collections (best choice):
From `java.util.concurrent` package:
 - ConcurrentHashMap
 - CopyOnWriteArrayList
 - CopyOnWriteArraySet

These are thread-safe and faster because they use better locking techniques.

Conclusion:

Use concurrent collections like ConcurrentHashMap for better performance and thread safety.

Comparison between Vector and ArrayList:

- Both are dynamic arrays in Java.
- Vector is thread-safe because all its methods are synchronized.
- ArrayList is not thread-safe.
- Vector is slower due to synchronization.
- ArrayList is faster and more commonly used.
- Vector is considered legacy (old), while ArrayList is modern.

Conclusion:

Use ArrayList in most cases. Use Vector only if you need thread safety (but better options exist).

Comparison between ArrayDeque and Stack:

- Both can be used as a stack (LIFO).
- Stack is a legacy class and extends Vector.
- Stack is thread-safe but slower.
- ArrayDeque is not thread-safe but much faster.
- ArrayDeque is more flexible because it can work as both stack and queue.

Conclusion:

ArrayDeque is preferred over Stack in modern Java because it is faster and more flexible.